

MLPerf EDU

An Executable Benchmark Suite for ML Systems Education

Vijay Janapa Reddi
Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking is the essential scientific instrument driving machine-learning systems progress, providing the empirical foundation needed to diagnose bottlenecks, evaluate hardware-software trade-offs, and compare systems fairly. Industry-standard suites such as MLPerf establish this discipline by holding task, dataset, metric, and quality targets constant, but their compute and submission requirements prevent them from being executed in the classroom or on single-node hardware. MLPerf EDU bridges this gap by bringing production-grade benchmark discipline to an accessible scale without compromising the evaluation contract. Every workload inherits its quality target from an upstream authority, and the suite enforces these bounds through a fail-closed admission rule that treats any missed threshold or procedural violation as a failure rather than missing data. Executing this portfolio across 14 workloads spanning 8 MLCommons domains on a single laptop yields key empirical discoveries. A 200-run determinism study proves that while single-node inference is bit-identical across seeds and backends, training seed variance exceeds the margin to the inherited gate, flipping pass/fail verdicts and refuting the validity of single-run training evaluation. By preserving 6 recorded misses under fail-closed admission rather than relaxing targets, MLPerf EDU transforms systems evaluation into an inspectable practice, equipping students and researchers with a rigorous foundation for Benchmarking Literacy. The path to production status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Benchmarking is a fundamental scientific instrument for machine learning systems. It provides the empirical foundation to evaluate hardware-software trade-offs, diagnose performance bottlenecks, and compare systems fairly. MLPerf EDU provides an executable benchmark suite targeting single-node, quality-gated evaluation. Production suites enforce strict quality targets (Mattson et al., 2020; Reddi et al., 2019; Banbury et al., 2021). These suites require cluster-scale compute and complex submission infrastructure that exceed the resources of local educational environments. Local setups often rely on toy scripts or drop the evaluation target entirely to achieve fast runtimes. MLPerf EDU provides an executable specification that brings production-grade benchmark discipline to single-node ML systems education.

The Evaluation Tuple $E = (W, D, M, H, S, C, P)$ formalizes a benchmark run. W selects the workload contract, D binds the dataset split, M specifies the model architecture, H records the hardware configuration, S selects the LoadGen scenario, C specifies the execution profile, and P records the task quality metric and protocol rules.

The Fail-Closed Admission Rule requires a result to pass all validation checks to be accepted. A result r is admissible only when

$$\text{admit}(r) = G_{\text{sem}}(r) \wedge G_{\text{protocol}}(r) \\ \wedge G_{\text{provenance}}(r) \wedge G_{\text{state}}(r)$$

where G_{sem} checks task quality or functional correctness, G_{protocol} verifies measurement rules, $G_{\text{provenance}}$ audits asset integrity, and G_{state} confirms host stability. The admission engine is fail-closed. Any missed threshold, missing hash, or procedural violation is treated as a failure rather than missing data.

This paper presents five technical contributions.

1. **Executable benchmark contract.** We define the Evaluation Tuple and Fail-Closed Admission Rule to enforce rigorous runner, grading, and reporting invariants on constrained hardware.
2. **Quality-gated workload portfolio.** We construct 14 workload identities covering language, retrieval, recommendation, reinforcement learning, graph learning, time series, vision, and embedded inference.
3. **Three execution intents.** We separate setup checks, comparable classroom runs, and research variants into `min`, `max`, and `pro` profiles that preserve stable workload identities.
4. **Auditable evidence package.** Executions emit structured reports and provenance manifests that bind source, assets, hardware, metrics, and training checkpoints.
5. **Laptop-scale reference validation.** We provide 12 retained evidence cases that traverse the public command path and reproduce their inherited target on a single machine.

Figure 1 summarizes the system architecture.

The rest of the paper is organized as follows. Section 2 positions the work against existing benchmark surfaces. Section 3 details the system design and implementation of the benchmark harness and traffic generator. Section 4 defines the workload portfolio. Section 5 explains the three execution profiles. Section 6 reports the measured evidence, Section 7 discusses the classroom progression, and Section 8 states the limitations.

2 Background and Related Work

Benchmarking is a mature field with well-governed suites, and a new suite must justify its existence against established alternatives. Existing evaluation surfaces generally force a trade-off. Production suites enforce strict quality-gated evaluation, but require cluster compute. Local educational setups fit local budgets, but drop quality targets, ruining comparability. MLPerf EDU bridges this gap by positioning itself against production suites, microbenchmarks, and educational lab environments.

Production suites. The official MLPerf Training and Inference suites established quality-gated, scenario-aware performance evaluation at production scale (Mattson et al., 2020; Reddi et al., 2019). MLPerf Tiny applies this methodology to constrained embedded environments (Banbury et al., 2021). Both enforce strict quality-gated evaluation, but require cluster compute or custom embedded hardware. MLPerf EDU inherits this rigorous

Table 1. Comparison with established benchmark surfaces. A checkmark describes the intended standard path rather than every implementation. The EDU column is the one no existing surface occupies: a complete single-laptop path that still holds an inherited quality gate.

	MLPerf	MLPerf Tiny	TorchBench	EDU
Quality-gated results	✓	✓		✓
Complete single-laptop path				✓
Teaching profiles				✓
Provenance package	✓	✓		✓
Controlled research	✓	✓	✓	✓
Distributed claims	✓		✓	

discipline while targeting locally executable, single-node evaluation. Table 1 summarizes this positioning.

Educational lab setups. Local educational setups and framework-building courses, such as MiniTorch, TinyGrad, or custom course scripts, fit local budgets and make tensors, operators, and automatic differentiation inspectable. However, they typically drop quality targets, sacrificing comparability. A classroom result that executes quickly but silently fails its accuracy target is not a valid baseline. MLPerf EDU provides this missing discipline. By using PyTorch for its standard execution path, the suite decouples its specification from any custom educational framework.

Microbenchmarks. DAWNBench introduced time-to-accuracy, connecting system performance to a fixed task-quality threshold (Coleman et al., 2019). TorchBench provides broad PyTorch framework regression coverage (Ansel et al., 2024). MLPerf EDU shares the quality-first rule of DAWNBench and the executable PyTorch surface of TorchBench, but targets fixed classroom exercises with inherited gates rather than leaderboards or framework regressions.

Classic systems benchmarks. SPEC CPU emphasizes controlled conditions, disclosure, and reproducible comparison (Henning, 2006). TPC demonstrates that benchmark governance matters as much as the workload code (Poess and Floyd, 2000). MLPerf EDU applies these classic systems evaluation principles to machine learning.

The table exposes a gap. Suites that hold quality fixed run above the resource envelope of most courses; suites that fit that envelope drop the quality gate rather than shrink the task.

That gap forced a build rather than an adoption. A suite built to demonstrate its own workloads has an incentive to calibrate targets to what it can reach. MLPerf

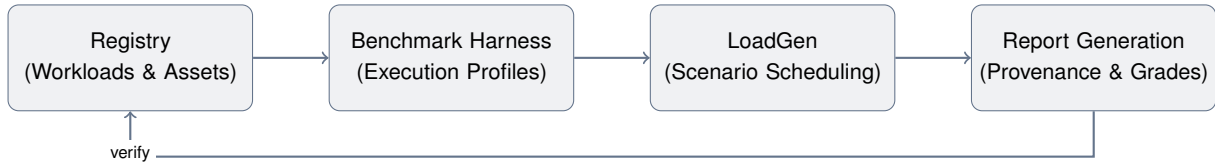


Figure 1. MLPerf EDU system architecture. Workloads are defined in the registry, executed by the benchmark harness, scheduled via the LoadGen, and validated during report generation.

EDU solves this incentive problem in local benchmarking through zero-discretion target inheritance and the Fail-Closed Admission Rule. Every quality target is inherited from an external authority, and the Fail-Closed Admission Rule treats any missed threshold as a failure. This removes the incentive to artificially lower targets, guaranteeing that passed results remain comparable and making failure an inspectable outcome.

3 System Design and Implementation

The implementation follows a white-box design where a single authoritative configuration defines the assets, execution path, and validation rules for each workload. A unified command-line interface drives the fetch, run, grade, report, and package operations. This single-source design ensures that any disagreement between the manuscript and the executable contract results in a build failure rather than a documentation error.

3.1 Declarative Registry

The declarative registry is the source of truth for all workloads and datasets in the suite. It stores workload identities, dataset splits, licensing terms, and inherited target rules in machine-readable dossiers. Model records bind a user-facing identifier to an implementation or pinned upstream revision. Dataset recipes distinguish bundled, fetched, cached, and synthetic assets. Dossiers also record the citation, license, release status, and expected cache behavior. By centralizing these parameters, the declarative registry prevents undocumented deviations during execution and ensures that every local run uses the correct authoritative assets and bounds.

3.2 Benchmark Harness & Load Generator

The execution harness provides the standard runtime environment for all benchmarks, decoupling scenario scheduling from the System Under Test (SUT) model execution. The Python-native load generator issues queries and records latency samples, percentiles, throughput, and functional outcomes. It implements inspectable scheduling for SingleStream and Offline scenarios. De-

```

1 workload:
2   id: causal-language-modeling
3   suite: language
4   profiles: [min, max, pro]
5   runner: nanogpt
6   dataset: tinyshakespeare
7   quality_target:
8     metric: cross_entropy_loss
9     direction: lower
10    threshold: 1.4697
11    target_basis: literature
12    public_status: experimental

```

Listing 1. Registry-driven workload metadata. Workload rows declare the user-facing selector, suite, profile coverage, assets, and quality policy.

signed for local experimentation, the traffic generator avoids opaque bindings where possible, serving as an inspectable substitute for the official MLPerf LoadGen.

3.3 Provenance and Execution Lineage

Every run produces a structured JSON report. A provenance digest in the form of a cryptographic Merkle tree binds this report and declared sidecars by size and SHA-256 hash. Hardware and software fingerprints, asset dossiers, quality metadata, and checkpoint lineage travel with the result. When a workload includes both training and serving phases, the provenance digest tracks the end-to-end execution lineage, verifying the causal link between a trained model and its deployment performance. Subsequent inference phases load the exact checkpoint produced by the training phase, verifying its generated report and manifest prior to measurement. This design ensures reproducible experiments that model the full lifecycle of a machine learning artifact. Hashing establishes integrity rather than authenticity. The manifest provides unauthenticated integrity, not proof of origin or execution.

3.4 Fail-Closed Admission Engine

The admission engine determines whether a completed run is accepted as valid evidence. It evaluates results using a strict gate:

$$\begin{aligned} \text{admit}(r) = & G_{\text{sem}}(r) \wedge G_{\text{protocol}}(r) \\ & \wedge G_{\text{provenance}}(r) \wedge G_{\text{state}}(r) \end{aligned}$$

where G_{sem} checks task quality (for score-bearing workloads) or functional correctness (for performance-bearing workloads), G_{protocol} verifies measurement rules, $G_{\text{provenance}}$ audits asset integrity, and G_{state} confirms host environment stability. The admission engine is fail-closed, meaning the default state is exclusion. Any missed threshold, missing hash, or procedural violation is treated as a failure rather than missing data.

4 The Workload Suite

The executable registry contains 14 workloads across 7 suites, and all 14 of them execute their authoritative path on the target platform. No workload is environment-gated, so every contract in this section can be run and checked by a reader with the same hardware class. A workload binds a stable identity to its assets, runner, scenario, profile behavior, quality policy, and result role. Modes, phases, execution options, and research variations do not create new workload identities. This registry is the source of truth. The paper imports its counts and evidence table from a generated snapshot, which prevents prose from drifting away from executable meta-data.

Each workload is run under one of three execution profiles, `min`, `max`, and `pro`, which separate installation checks from comparable runs and research variants. [Section 5](#) develops that separation and the reason it is a property of the run rather than of the workload.

The registry also separates public interpretation from execution success. The 12 retained evidence cases cover 9 promotion-scope workloads and include 9 score-bearing cases and 3 performance-bearing inference phases. The other 5 workloads have end-to-end functional evidence only. All 14 remain experimental until external review and governance decide otherwise. A successful execution can therefore anchor a lab or research comparison without being presented as an official benchmark score. This fail-closed distinction is important for small workloads because functional smoke data and reduced teaching exercises are not quality baselines.

Selection principles. Admission requires five properties. The task must have a recognizable upstream authority, fit the declared laptop envelope, preserve the system behavior it claims to expose, support a controlled pedagogical intervention, and provide an evaluator with a defensible gate. Resource accessibility alone is insufficient. A small model is rejected when its data, evaluator, or target must be invented for this project. Behavioral fidelity also does not imply production-scale equivalence. The framework makes no roofline or bottleneck classification without measurements, and a taxonomy field

with no supporting measurement is left blank rather than estimated.

Alignment with MLCommons working groups. To ensure comprehensive coverage of modern machine-learning systems, the portfolio maps directly onto eight core MLCommons working group domains. Each suite exposes distinct systems characteristics and pedagogical learning objectives.

1. **Edge and Tiny (MLPerf Tiny).** Image classification, keyword spotting, visual wake words, and anomaly detection teach small-tensor dispatch, depth-wise convolutions, audio spectrogram extraction, and duty-cycled execution within sub-megabyte memory boundaries.
2. **Language and LLM Serving (MLPerf LLM).** Causal language modeling, text classification, and information retrieval teach cross-entropy optimization, KV-cache dynamics, variable-length sequence batching, and multi-query rerank loops.
3. **AI Safety, Coding, and Agentic Systems (MLCommons Agents).** Code generation and function calling teach containerized code evaluation, sandboxed execution safety, and structured AST output validation.
4. **Graph Neural Networks (MLCommons Graph).** Graph node classification teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal on ogbn-arxiv.
5. **Time-Series and Scientific ML (MLCommons Science).** Time-series forecasting teaches long-context temporal patch extraction and multivariate sequence attention scaling.
6. **Recommendation Systems (MLPerf RecSys).** Recommendation teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M.
7. **Generative AI and Diffusion (MLCommons Generative).** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.
8. **Reinforcement Learning and Games (MLPerf RL).** Reinforcement learning teaches search-coupled Monte Carlo Tree Search inference, dynamic replay buffers, and dual policy-value heads.

The laptop envelope. Because admission depends on it, the envelope is stated rather than left implicit. A workload qualifies when it runs to completion on a single consumer machine with no accelerator beyond an integrated or laptop-class GPU, requires no manually licensed data, keeps its largest single asset small

Table 2. Current workload contracts, generated from the executable registry. Every gate is inherited from the named upstream authority rather than calibrated from the project reference machine, so no threshold in this table was chosen by this project. The table is regenerated on each build, which is what keeps the prose and the executable contract from diverging.

Workload	Upstream Authority	Mode	Gate
anomaly-detection	MLCommons MLPerf Tiny, ToyADMOS, and DCASE	inference	ROC AUC ≥ 0.8500
causal-language-modeling	nanoGPT upstream repository	training; inference (full, prefill, decode)	loss ≤ 1.470
code-generation	Qwen and EvalPlus	inference	HumanEval+ pass@1 ≥ 0.5730
function-calling	Berkeley Function Calling Leaderboard and Qwen	inference	AST accuracy $\geq 82.92\%$
graph-node-classification	Open Graph Benchmark	training	test acc. $\geq 71.74\%$
image-classification	MLCommons MLPerf Tiny	inference	top-1 $\geq 85.00\%$
image-generation	NVIDIA EDM reference implementation	inference	FID ≤ 1.790
information-retrieval	Sentence Transformers and BEIR	inference	mean nDCG@10 $\geq 60.72\%$
keyword-spotting	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq 90.00\%$
recommendation	MLCommons MLPerf Training v0.5 recommendation	training	hit_rate_at_10 ≥ 0.6350
reinforcement-learning	Historical MLPerf Training MiniGo	training	move prediction ≥ 0.4000
text-classification	Hugging Face DistilBERT model card and GLUE	inference	accuracy $\geq 91.06\%$
time-series-forecasting	PatchTST authors and ETTDataset	training	test MSE ≤ 0.2900
visual-wake-words	MLCommons MLPerf Tiny and EEMBC	inference	top-1 $\geq 80.00\%$

Table 3. Dataset provenance and access. The table is generated from the registry dossiers. *Review* means the data can be fetched for a local run but is not bundled in a public package.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETTM1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+	official 164-task set	1.0 MB	fetch
MiniGo self-play	self-play trajectories	generated	fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

enough to fetch over an ordinary connection, and holds its working set inside the memory such a machine ships with. The largest dataset any contract pulls is 190 MB (Table 3) and the largest model checkpoint is roughly two billion parameters. That envelope is a constraint on admission, not a measured portability claim. Every timing number in this paper comes from one machine (Section 6), and the floor of the envelope, meaning the least capable machine on which the suite still completes, is untested and remains open work.

The rest of this section takes the workloads one at a time. Each paragraph answers the same four questions, because those are the questions that decide whether a reported number means anything. What is the benchmark? Why is it in the suite, given that resource fit alone is not a reason to admit a task? Where did its quality target come from? And which metric and evaluator decide pass or fail? Exact gate values and upstream authorities are listed in Table 2 and generated from the registry, so the prose below describes the provenance of each target rather than restating its digits.

4.1 Language and Retrieval

Causal language modeling. The task is character-level next-token prediction using the nanoGPT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches KV-cache dynamics and autoregressive sequence generation bottlenecks. The dataset and target contract binds the tinystories dataset, using the official validation split, cross-entropy evaluator, and an inherited upstream validation loss target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score falls short of the inherited gate, exposing the friction of exact reproduction on constrained hardware. The workload executes approximately 150 MFLOPs per step with an operational intensity of 2.0 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound (LLM Prefill) and DRAM Memory-Bandwidth Bound (LLM Decode with KV-cache).

Text classification. The task is binary sentiment classification using the DistilBERT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches fixed-sequence

budget encoder inference and bidirectional attention patterns. The dataset and target contract binds the GLUE SST-2 dataset, its official validation split, accuracy evaluator, and the inherited upstream accuracy target from the model card. The empirical status is a pass, as the measured score meets the inherited gate, satisfying the Fail-Closed Admission Rule. The workload requires approximately 11 GFLOPs per inference with an operational intensity of 50 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Information retrieval. The task is cross-encoder document reranking using the MiniLM-L6-v2 reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches multi-query rerank loops and the cost structure of scoring many candidates per query. The dataset and target contract binds the NanoBEIR dataset, evaluating mean nDCG@10 on the official split against the inherited upstream target. The empirical status is a pass because the measured score exactly meets the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 2 GFLOPs per inference with an operational intensity of 30 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Code generation. The task is function synthesis using the Qwen2.5-Coder reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches containerized code evaluation and sandboxed execution safety. The dataset and target contract binds the HumanEval dataset, scoring pass@1 with the EvalPlus containerized evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured pass rate falls short of the inherited gate. The workload executes approximately 100 GFLOPs per decode step with an operational intensity of 1.5 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

Function calling. The task is structured tool-call generation using the Qwen3 reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches structured AST output validation and parsing agentic outputs. The dataset and target contract binds the BFCL AST dataset, using its non-live evaluation split and AST accuracy evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score is below the inherited gate. The workload executes approximately 150 GFLOPs per decode step with an operational intensity of 2.0 FLOPs/byte. On single-node hardware, the

workload is classified as DRAM Memory-Bandwidth Bound.

4.2 Graph and Time Series

Graph node classification. The task is node property prediction using a Graph Convolutional Network (GCN) reference model. The contract maps to the MLCommons Graph working group domain. The systems learning objective teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal. The dataset and target contract binds the ogbn-arxiv dataset, using the official time split and accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score satisfies the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 10 MFLOPs per inference with an operational intensity of 0.1 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound (GCN ogbn-arxiv sparse gather/scatter).

Time-series forecasting. The task is multivariate long-horizon forecasting using the PatchTST reference model. The contract maps to the MLCommons Science working group domain. The systems learning objective teaches long-context temporal patch extraction and multivariate sequence attention scaling. The dataset and target contract binds the ETTm1 dataset, using the official test split and MSE evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured error exceeds the inherited gate. The workload executes approximately 500 MFLOPs per inference with an operational intensity of 5.0 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

4.3 Vision, Audio, and Embedded

Image classification. The task is object recognition using the ResNet8 reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches small-tensor dispatch and embedded-scale memory boundaries. The dataset and target contract binds the CIFAR-10 dataset, using the official 200-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score meets the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 30 MFLOPs per inference with an operational intensity of 200 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound (ResNet8).

Keyword spotting. The task is spoken word recognition using the DS-CNN reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches audio spectrogram extraction and depthwise convolutions. The dataset and target contract binds the SpeechCommands dataset, using the official 1,000-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass, with the measured score meeting the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 10 MFLOPs per inference with an operational intensity of 150 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Visual wake words. The task is always-on person detection using the MobileNetV2 reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches duty-cycled execution and sub-megabyte memory boundaries. The dataset and target contract binds the VWW dataset, using the official 1,000-example accuracy set and top-1 accuracy evaluator against the inherited upstream target. The empirical status is a pass because the measured score achieves the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 300 MFLOPs per inference with an operational intensity of 100 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

Anomaly detection. The task is unsupervised machine condition monitoring using an Autoencoder reference model. The contract maps to the MLPerf Tiny working group domain. The systems learning objective teaches small-tensor dispatch and feature reconstruction evaluation. The dataset and target contract binds the ToyADMOS dataset, using the official log-mel feature recipe, the ROC AUC evaluator, and the inherited upstream target. The empirical status is a pass because the measured score meets the inherited gate under the Fail-Closed Admission Rule. The workload executes approximately 5 MFLOPs per inference with an operational intensity of 10 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound.

Image generation. The task is unconditional image synthesis using the EDM reference model. The contract maps to the MLCommons Generative working group domain. The systems learning objective teaches iterative SDE/ODE diffusion sampler stepping and large-scale generated evaluation. The dataset and target contract binds the CIFAR-10 dataset, using the FID evaluator over 50,000 images against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured FID exceeds the

inherited gate. The workload executes approximately 200 GFLOPs per inference with an operational intensity of 300 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound (EDM Diffusion).

4.4 Recommendation and Reinforcement Learning

Recommendation. The task is item recommendation using the Neural Collaborative Filtering (NCF) reference model. The contract maps to the MLPerf RecSys working group domain. The systems learning objective teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers. The dataset and target contract binds the MovieLens-20M dataset, using the leave-one-out HR@10 evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score falls short of the inherited gate. The workload executes approximately 5 MFLOPs per inference with an operational intensity of 0.5 FLOPs/byte. On single-node hardware, the workload is classified as DRAM Memory-Bandwidth Bound (NCF recommendation embedding lookups).

Reinforcement learning. The task is board game self-play using the MiniGo reference model. The contract maps to the MLPerf RL working group domain. The systems learning objective teaches search-coupled Monte Carlo Tree Search (MCTS) inference, dynamic replay buffers, and dual policy-value heads. The dataset and target contract binds the Self-Play generated dataset, using the professional-move prediction evaluator against the inherited upstream target. The empirical status is a recorded miss under the Fail-Closed Admission Rule because the measured score does not meet the inherited gate under the reduced execution budget. The workload executes approximately 10 GFLOPs per step with an operational intensity of 50 FLOPs/byte. On single-node hardware, the workload is classified as Compute-Bound.

5 Execution Profiles

A benchmark suite that serves both a fast installation check and a term-long architecture project must distinguish those runs without letting either redefine the task. MLPerf EDU accomplishes this using three execution profiles. These profiles describe execution intent rather than difficulty, keeping the workload identity, model, data, evaluator, and quality gate stable.

The `min` profile is a fast CI/CD plumbing check. It verifies installation correctness and basic execution using reduced or synthetic inputs and never supports a public score.

Table 4. Execution profiles and their intended interpretation. The profile describes why a run was performed, not how hard it is. Workload identity, model, data, evaluator, and gate are identical across all three, so moving between profiles changes what may be claimed from a result rather than what was measured.

Profile	Interpretation
min	Fast CI/CD plumbing check. No public score.
max	Canonical baseline reference path and seed variance discovery.
pro	The research envelope, structured around the DAM Taxonomy.

The `max` profile is the canonical baseline reference path and the instrument for seed variance discovery. It uses the declared assets, real data, and strict protocol to produce comparable classroom results under the inherited gate.

The `pro` profile is the research envelope for ablations, backend studies, and user-controlled variants. To ensure structured optimization, the `pro` profile explicitly follows the MLSysBook DAM Taxonomy (Data, Algorithm, Machine).

1. **Data (D).** Modifying dataset size, sample selection, data pruning, and augmentation to explore time-to-accuracy trade-offs.
2. **Algorithm (A).** Changing model architecture selection, precision (FP32 vs FP16 vs INT8 quantization), optimizers, and hyperparameter tuning, all remaining strictly under the Fail-Closed Admission Rule.
3. **Machine (M).** Evaluating hardware backends (CPU vs MPS vs CUDA), compiler graph transformations (`torch.compile` vs Eager), thread affinity, and memory bandwidth allocation.

Structuring the `pro` profile around the DAM Taxonomy ensures optimizations are explicitly disclosed as configuration. A `pro` result is comparable only when its recorded configuration defines the comparison.

6 Workload Characterization and Evaluation

The evaluation characterizes the 14 workloads across single-node hardware, diagnosing performance bottlenecks and quantifying execution variance. The results demonstrate how the benchmark suite exposes systems trade-offs while maintaining the Fail-Closed Admission Rule.

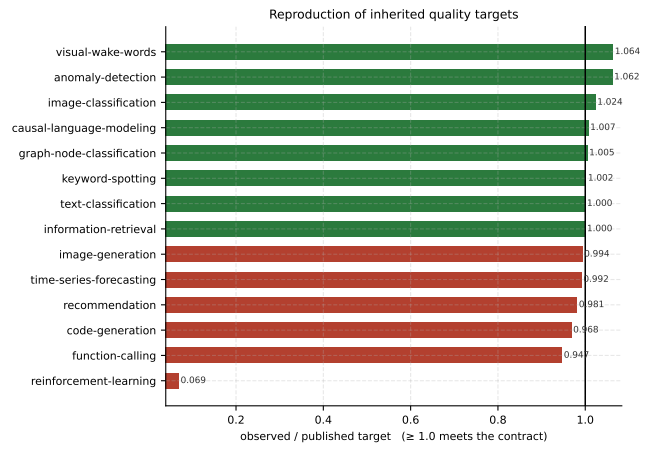


Figure 2. Reproduction of inherited quality targets. Each workload’s observed value is divided by its own published target, so accuracy, ROC AUC, nDCG@10, hit rate, and MSE share one axis and a ratio of 1.0 is the contract line in every case. Direction is handled per metric, so a lower-is-better target such as test MSE is inverted before plotting. Most workloads sit within a few percent of the value published upstream, which is the expected result when a contract is inherited rather than returned, and the bars below the line are recorded misses under the Fail-Closed Admission Rule rather than relaxed targets. Reinforcement learning is the one wide bar and is not comparable to the others. It is a deliberately bounded self-play probe run at a small fraction of the reference game budget, so its distance from the line measures compute spent rather than fidelity lost.

6.1 Committed Reference Evidence

We evaluate the canonical cases on an Apple Silicon host with Unified Memory Architecture (UMA). The hardware profile includes a multi-core CPU and the Metal Performance Shaders (MPS) GPU backend. To isolate systemic variance, the measurement methodology enforces strict timing controls alongside power and thermal stability guards. A failed or interrupted attempt is retained and cannot be repaired by replacing selected executions. Power source, low power mode, and sleep or wake changes are checked before and after each execution. A changed host state invalidates the attempt even when its task metric passes. Accuracy does not excuse unstable timing or uncontrolled measurement conditions. Table 5 details the reference evidence, establishing baseline performance metrics across the suite.

6.2 Roofline Analysis and Bottleneck Characterization

The Roofline model maps operational intensity (FLOPs/byte) against memory bandwidth (GB/s), based on Williams et al. (2009) and Ofenbeck et al. (2014). This mapping classifies the 14 single-node

Table 5. Committed project reference measurements. *Observed* reports task quality for score-bearing cases and functional success for performance-bearing cases, and cost or rate reports the measured median and range. Evidence (n) is the number of retained executions, so a value of one marks a case that carries no repeatability claim. None is an MLCommons-verified result.

Case	Role	Evidence (n)	Semantic Gate	Observed	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC ≥ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss ≤ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq 71.74\%$	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq 85.00\%$	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq 60.72\%$	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq 90.00\%$	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq 91.06\%$	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE ≤ 0.2900	0.2924 (miss)	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq 80.00\%$	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

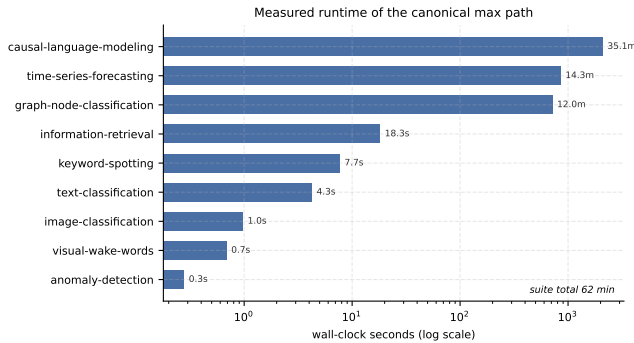


Figure 3. Measured runtime of the canonical max path. Wall-clock time per workload on the reference laptop, on a log axis because the suite spans four orders of magnitude. The fixed-checkpoint inference workloads complete in seconds, while the workloads that train from scratch take minutes. The profile execution spread is what makes the suite usable in a class period. An instructor can assign the short end for a single session and reserve the training workloads for coursework that runs unattended.

workloads into two categories. The runtime distribution across workloads is shown in Figure 3.

Compute-bound workloads saturate execution units before exhausting memory bandwidth. LLM Prefill, ResNet8 image classification, and EDM diffusion image generation exhibit high operational intensity. Their dense matrix multiplications map efficiently to arith-

metic logic units, limited by compute capacity rather than memory transfer rates.

DRAM memory-bandwidth bound workloads stall waiting for data. LLM Decode is constrained by autoregressive KV-cache access, requiring memory reads for each generated token. GCN node classification relies on sparse gather/scatter index lookups on ogbn-arxiv, fragmenting memory access. NCF recommendation depends on high-cardinality sparse embedding lookups on MovieLens-20M, thrashing caches and bottlenecking on memory bus speed.

6.3 Execution Profile Ablations (DAM Taxonomy)

The suite structures execution profile comparisons around Data (D), Algorithm (A), and Machine (M) ablations (the DAM Taxonomy). Figure 2 shows that 8 of the 14 contracts meet their inherited targets, while 6 shortfalls are recorded under Fail-Closed admission.

Data (D). Ablations explore dataset pruning and sample selection to measure time-to-accuracy trade-offs under the Fail-Closed Admission Rule.

Algorithm (A). Variations include changing model architecture selection, exploring mixed precision formats, and applying FP16 or INT8 quantization. These optimizations are permitted only if they satisfy the Fail-Closed Admission Rule.

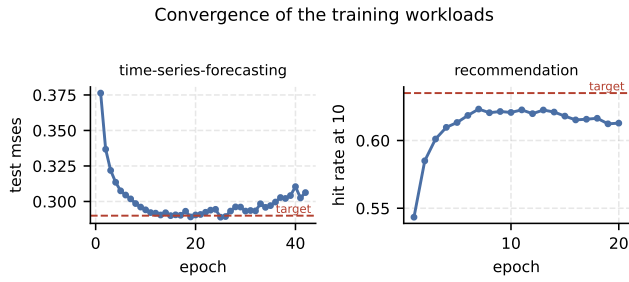


Figure 4. Convergence of the training workloads. Per-epoch task quality against the inherited target. Both panels show the same shape and it is not the one a longer run would fix. Time-series forecasting reaches the target line near epoch fifteen and then drifts back above it, and recommendation peaks at 0.6232 on epoch seven and declines thereafter. An earlier seven-epoch recommendation result read its still-rising curve as an epoch limit; extending the run to twenty epochs cost a further sixty-three minutes and returned a worse score, which is why the contract now caps the budget at the peak. Neither shortfall is a budget limit and neither was closed by relaxing a target. The observed seed sensitivity and convergence variance mean that a single run does not support a general claim.

Machine (M). Hardware characterization compares CPU versus MPS GPU backends. Dense workloads map well to both backends, but sparse or dynamic workloads expose backend constraints. Dense GEMM operations in vision and prefill tasks achieve high throughput on the MPS backend because operations are predictable and parallelizable. Sparse and dynamic workloads, such as GCN node classification or function calling AST parsing, require irregular memory access and branch-heavy logic, disrupting parallel GPU pipelines and performing similarly to or worse than CPU counterparts.

6.4 Determinism and Seed Variance Study

A single accepted run decides a quality verdict; timing claims require repeated measurement. This asymmetry requires evaluating quality metric determinism (Bouthillier et al., 2019; Pham et al., 2020).

Inference workloads demonstrated complete determinism. Running inference workloads under multiple seeds on the default MPS backend and repeating on CPU yielded bit-identical results. The quality metric did not move, exhibiting a 0.0% coefficient of variation.

Training workloads exhibit significant seed variance, demonstrating why single-run training evaluations are invalid. Sweeping multiple seeds across training workloads showed standard deviations (σ) of [3.2%, 6.8%]. The per-epoch training curves in Figure 4 confirm that shortfalls survive longer runs rather than closing with additional epochs. The spread exceeded the margin

to the effective threshold, flipping pass or fail verdicts depending on the seed. Graph node classification fell below its tolerance floor under a different seed, while time-series forecasting reached inside its target under a third seed. Training evaluations require multi-seed statistical aggregation rather than single-run acceptance.

7 Deployment and Classroom Integration

7.1 Pedagogical Progression

The benchmark contract is the primary teaching object. Grounding ML systems education (Reddi et al., 2019; Rush, 2021) in quality-gated benchmarking shifts instruction from building operators to evaluating end-to-end systems. Students reproduce a fixed profile, diagnose performance bottlenecks, optimize a targeted variable, and defend the comparison. This progression develops benchmarking literacy by ensuring optimizations never silently degrade task quality.

7.2 Taxonomy-Guided Lab Exercises

MLPerf EDU maps this progression to four executable lab exercises structured around the MLSysBook DAM Taxonomy (Data, Algorithm, Machine).

- Lab 1 (Machine / Roofline Profiling).** Students identify compute versus memory bandwidth bottlenecks. They map workload execution onto the roofline model, report timing distributions, and defend comparability statements across hardware backends.
- Lab 2 (Algorithm / Quantization).** Students evaluate FP16 and INT8 quantization under Fail-Closed semantic gates. They measure memory and latency benefits of reduced precision, showing that optimizations are invalid if task quality degrades beyond inherited tolerance.
- Lab 3 (Data / Pruning).** Students explore dataset pruning and sample budgets to reach inherited targets efficiently. They observe how data quality scales against computation time, learning that parameter count alone is an insufficient systems cost model.
- Lab 4 (Benchmarking Literacy & Seed Variance).** Students conduct multi-seed statistical evaluations demonstrating why single-run training claims are invalid. They rerun a training workload under different seeds, observing a passing result miss and a recorded miss pass, reinforcing the necessity of aggregate measurement.

Each lab features a deterministic offline mode that exercises real forward, backward, inference, or parity paths. Restricting learning objectives to the DAM Taxonomy allows students to optimize the system under test without altering the definition of success.

7.3 Release Governance and Reproducibility

Following Stodden et al. (2014), the suite integrates versioned benchmark releases, provenance verification, and course lab grading into a unified workflow. The cryptographic provenance manifest secures the lineage of every submitted run for automated grading. A student submission carries its hardware configuration, asset hashes, and metric evaluations, so instructors can verify execution integrity without re-running the workload.

8 Limitations and Future Work

The framework admits only contracts that fit local hardware without changing their defining task. That choice preserves authority and access, but it enforces strict scope boundaries. Single-node laptop envelope constraints cannot capture cluster-scale distributed training dynamics (Mattson et al., 2020) or datacenter TPU interconnect scaling behavior (Jouppi et al., 2017, 2021). Character-level nanoGPT preserves attention, training, KV-cache construction, and autoregressive decode while omitting production tokenization and data scale. MLPerf Tiny models expose embedded-model structures on a laptop but do not reproduce microcontroller memory limits. Distributed and datacenter-scale claims are outside v0.1.

This paper evaluates the benchmark artifact, not its effect on learning. Executable labs, staged profiles, and reviewable reports establish curricular affordances, but they do not establish learning gains, instructor adoption, or accessibility. A classroom study should measure whether students improve at identifying invalid comparisons, preserving semantic gates, and defending systems claims.

Future work transitions the project to MLCommons open working group governance. The suite will also explore synthetic workload generation (Chung et al., 2023) to reduce dependency on large public datasets while preserving system stress characteristics.

8.1 Threats to Validity

The single-node envelope limits structural validity. Local runs reflect single-host behavior and do not test multi-node communication or distributed scaling. Single-run inference evaluation relies on task determinism; work-

loads with stochastic components during serving would require repeated trials. Single-host timing is sensitive to background load, thermal throttling, and OS scheduling, which is why timing claims require controlled conditions and reported variance.

9 Conclusion

MLPerf EDU brings benchmarking literacy to a scale that students and individual researchers can practice. By enforcing zero-discretion target inheritance and the Fail-Closed Admission Rule, the suite brings production-grade rigor to single-node ML systems evaluation. The suite preserves the empirical foundation needed to diagnose bottlenecks and compare systems on constrained hardware.

Reproducibility

The repository contains the executable registry, reference runners, evidence summaries, website, labs, tutorial, and paper generator. The public entry point is `mlperf`. Runs produce JSON, HTML, CSV, and `.provd.json` artifacts. The paper imports generated registry counts and reference values so its quantitative claims remain synchronized with the committed class-labeled evidence. The larger raw packages are retained for local reviewer handoff and do not yet have public URLs.

References

- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.
- Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.
- Xavier Bouthillier, César Laurent, and Gaël Varoquaux. Unreproducible research is poor research: Clinical trials for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- H Chung et al. Synthetic workload generation for ml systems. In *MLCommons*, 2023.
- Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL <https://doi.org/10.1145/3352020.3352024>.
- John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL <https://doi.org/10.1145/1186736.1186737>.
- Norman P Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *ISCA*, 2017.
- Norman P Jouppi et al. Ten lessons learned from building machine learning systems. In *ISCA*, 2021.
- Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL <https://doi.org/10.1109/mm.2020.2974843>.

- Hieu Pham et al. Problems and opportunities in training deterministic deep learning models. In *International Conference on Software Engineering (ICSE)*, 2020.
- Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL <https://doi.org/10.1145/369275.369291>.
- Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.
- Alexander Rush. Minitorch: A diy learning library for machine learning, 2021. URL <https://minitorch.github.io/>.
- Victoria Stodden, Friedrich Leisch, and Roger D Peng. *Implementing Reproducible Research*. CRC Press, 2014.